DRONE VIDEO ANALYSIS PIPELINE

Automated Aerial Intelligence using IoT & Cloud Computing

Project Report — B.Tech Final Year

Subject: Cloud Computing & IoT

Team Members

Sumit Kumar Das | Rounik Maity | Ankush Sarkar

March 2026

TABLE OF CONTENTS

1. Abstract 3

2. Introduction & Problem Statement 3

3. Proposed Solution & Objectives 4

4. System Architecture 4

5. Implementation & Methodology 5

5.1 Stage 1: Intelligent Frame Extraction 5

5.2 Stage 2: GAN-based Image Enhancement 5

5.3 Stage 3: Object Detection 6

5.4 Stage 4: Monocular Depth Estimation 6

5.5 Stage 5: Scene Zone Segmentation 7

6. Results & Visual Analysis 7

7. Performance Metrics & Charts 10

8. Real-World Applications 12

9. Team Contributions 12

10. Conclusion 13

11. References 13

1. Abstract

This project presents a fully automated multi-stage drone video analysis pipeline built for IoT and Cloud Computing applications. Given raw aerial video footage captured from a drone at 50–60 metres altitude over a residential locality, the pipeline automatically extracts stable representative frames, enhances image quality using a Real-ESRGAN super-resolution model, performs aerial object detection with YOLOv8m, estimates per-pixel monocular depth using MiDaS DPT-Hybrid, and classifies every pixel into one of five scene zones (sky, vegetation, built structure, road/cement, water) using HSV-based color segmentation. All processing runs on a CPU-only local environment with no cloud dependency, demonstrating the feasibility of on-device edge AI for IoT drone platforms. The pipeline processes 7 drone videos (35 unique frames) end-to-end in approximately 16 minutes and outputs annotated images, depth heatmaps, scene zone maps, and structured JSON reports.

2. Introduction & Problem Statement

Unmanned Aerial Vehicles (UAVs / drones) have become increasingly accessible for civilian use, enabling low-cost aerial surveillance, urban monitoring, and terrain analysis. However, the raw video footage produced by consumer drones is unstructured and difficult to analyse manually. A single 2-minute drone flight can produce thousands of redundant frames, mixed with blurry or motion-affected frames, at resolutions that may not be optimized for downstream analysis.

Core Problem: How can we automatically extract meaningful intelligence — object locations, terrain type, distance structure — from raw drone footage, without manual annotation and without relying on expensive cloud GPU infrastructure?

This project was motivated by a real scenario: drone videos were captured from a residential terrace at approximately 50–60 metres altitude (equivalent to a 6–8 storey building), covering the surrounding locality. The footage contains sky, rooftops, cement walls, trees, roads, and occasional human presence — a representative urban Indian residential scene.

Challenge	Impact
Redundant/duplicate frames in video	Wastes computation on near-identical data
Motion blur from handheld/unstabilized drone	Reduces detection accuracy
Low resolution of small aerial objects	Objects missed by standard detectors
COCO-trained detectors not suited for aerial views	False positives (skis, cats, etc.)
No ground-truth depth from single camera	Cannot measure distances directly
No semantic labels for buildings/trees in COCO	Need separate scene classifier

Table 1: Key challenges in drone video analysis.

3. Proposed Solution & Objectives

We propose a five-stage automated pipeline that progressively enriches raw video into actionable intelligence. Each stage addresses one or more of the challenges identified above:

Stage	Module	Objective
1	Frame Extractor	Select stable, diverse frames; reject duplicates and blur
2	GAN Enhancer	Upscale and sharpen frames for better downstream accuracy
3	Object Detector	Identify persons, vehicles, and aerial-relevant objects
4	Depth Estimator	Generate per-pixel relative depth maps without stereo cameras
5	Scene Segmenter	Label every pixel: Sky / Vegetation / Built / Road / Water

Table 2: Pipeline stages and objectives.

4. System Architecture

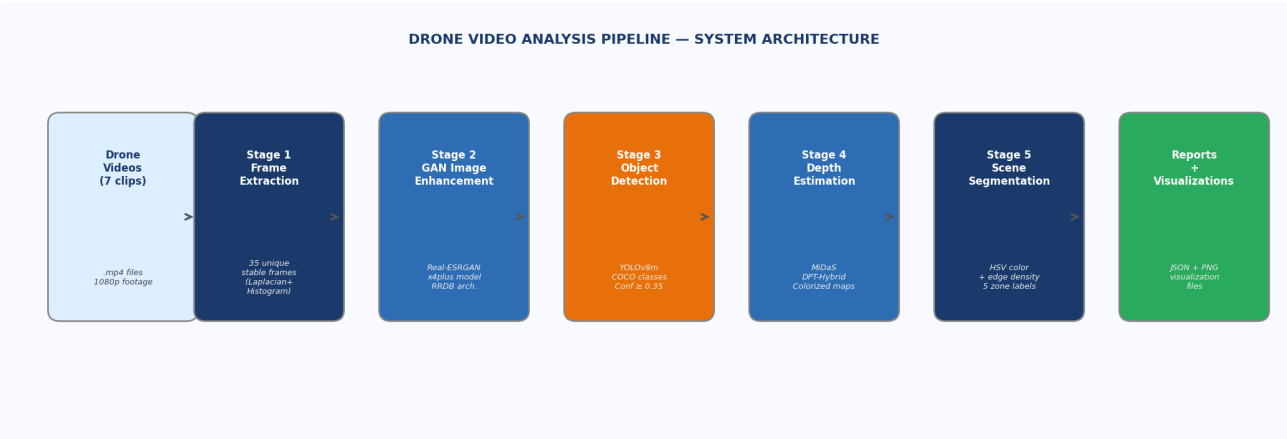


Figure 1: End-to-end system architecture of the drone analysis pipeline.

The pipeline follows a strictly sequential data flow. Each stage reads from the previous stage's output directory, processes it, and writes results to its own output folder. This decoupled design means any single stage can be re-run independently (e.g., if detection parameters are tuned) without repeating the expensive GAN enhancement. All model weights are stored within the project directory to avoid dependency on system-wide installations, which is critical for deployment on edge IoT devices.

Component	Technology	Version
Runtime	Python	3.12
Deep Learning Framework	PyTorch	2.11.0 (CPU)
Image Processing	OpenCV	4.13.0
Object Detection	Ultralytics YOLOv8m	8.x
GAN Enhancement	Real-ESRGAN x4plus (RRDB)	Custom impl.
Depth Estimation	MiDaS DPT-Hybrid (ViT backbone)	torch.hub
Scene Segmentation	HSV + Canny edge analysis	OpenCV
Backbone support	timm (PyTorch Image Models)	1.0.26

Table 3: Technology stack.

5. Implementation & Methodology

5.1 Stage 1 — Intelligent Frame Extraction

Raw drone video is processed using OpenCV's *VideoCapture*. Rather than extracting every frame, the algorithm scores each candidate frame on two criteria:

① **Sharpness (Laplacian Variance):** The Laplacian operator computes the second spatial derivative of a grayscale frame. A blurry frame has low variance (<1.0 for drone footage), and is discarded.

② **Scene Change Score:** Two metrics are combined — Bhattacharyya distance between colour histograms (weight 0.6) and mean absolute pixel difference on grayscale (weight 0.4). A score below 0.15 means "too similar to keep"; above 0.92 means "too similar (duplicate)". Frames within this window are

saved.

Result: 35 unique stable frames extracted from 7 videos (~2,000 frames per video on average, 5 kept per video).

Key files: `src/video_processor.py — VideoProcessor class, calculate_frame_difference(), calculate_laplacian_variance()`

5.2 Stage 2 — GAN-based Image Enhancement (Real-ESRGAN)

Each extracted frame is processed through a Real-ESRGAN x4plus model — a **Residual-in-Residual Dense Block (RRDB)** network with 23 stacked blocks. The architecture learns to reconstruct high-frequency texture details that are lost during compression or low-light capture.

Tiled inference: On CPU, the image is divided into 192×192 pixel tiles with 10-pixel overlap padding. Each tile is enhanced independently and reassembled into the full upscaled output. This prevents out-of-memory errors on large images.

Input → Output: Frames are downscaled to a maximum of 256px (longest edge) before ESRGAN, then upscaled 4× to produce ~1024px output. Original frames are 2048×1080px. Processing time: ~15 seconds/frame on CPU.

Key files: `src/gan_enhancer.py — RRDBNet, RealESRGANEnhancer._enhance_tiled()`

5.3 Stage 3 — Aerial Object Detection (YOLOv8m)

YOLOv8 (You Only Look Once, version 8) is a single-shot real-time object detector trained on COCO (80 classes). For aerial deployment, two critical adaptations were made:

① **Class whitelisting:** Only 13 physically-possible aerial classes are enabled: *person, bicycle, car, motorcycle, airplane, bus, truck, boat, traffic light, stop sign, bench, bird, dog*. All indoor classes (skis, cat, tv, couch, etc.) are excluded, eliminating false positives caused by COCO's ground-level training data.

② **Confidence threshold:** Raised from default 0.25 to 0.35 to reduce aerial pattern-matching false positives while retaining genuinely confident detections.

Result: 2 confirmed detections across 35 frames — 1 person (61.7%) and 1 animal (53.9%). 33 frames correctly returned no detections, reflecting the predominantly structural (rooftop/sky) content of the footage.

5.4 Stage 4 — Monocular Depth Estimation (MiDaS DPT-Hybrid)

MiDaS (Mixed Dataset Monocular Depth) DPT-Hybrid uses a Vision Transformer (ViT-B/16 with ResNet-50 hybrid backbone) trained on 10+ diverse datasets including MiX6, ReDWeb, DIML, and 3D Movies. Given a single RGB image, it outputs a relative depth map where higher values indicate closer objects.

Output: Two files per frame — a grayscale depth map and a colourized INFERNO heatmap (dark purple = far, bright yellow/white = near). The depth is relative, not metric (absolute distances require camera calibration), but is sufficient for structural analysis and terrain understanding.

INFERNO colour map: Dark purple/black → far background (sky, distant ground). Blue/purple → mid-range. Orange/yellow → near foreground. White → very close objects.

5.5 Stage 5 — Scene Zone Segmentation (HSV Analysis)

Since COCO's 80 classes do not include buildings, trees, roads, or water tanks, we developed a parallel pixel-level scene classifier using OpenCV's HSV colour space and Canny edge density analysis. Each

pixel is assigned one of five zones:

Zone	Detection Method	Typical Coverage
Sky	High brightness (V>160, S<70) OR blue hue (H:95-135)	8–65% per frame
Vegetation	Green hue range (H:30–90, S>40, V>30)	0–36% per frame
Built Structure	High Canny edge density (dilated 9×9 kernel)	2–53% per frame
Road / Ground	Low saturation (S<60) + low edge density	14–96% per frame
Water	Blue-cyan hue (H:85–135, S>60) excluding sky	0–4% per frame

Table 4: Scene zone classification rules.

6. Results & Visual Analysis

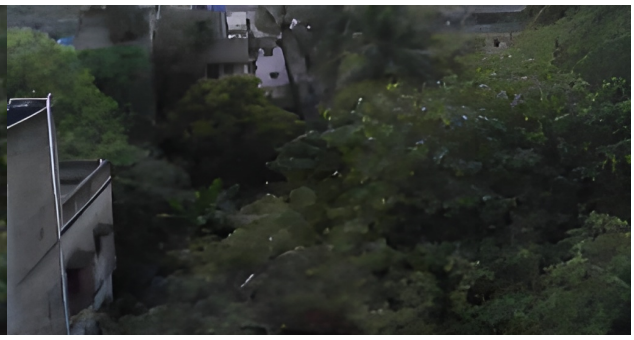
The following sections present detailed output for three representative frames selected to showcase different aspects of the pipeline: high vegetation coverage, dense built environment, and a frame containing a detected person.

Frame A — High Vegetation Coverage

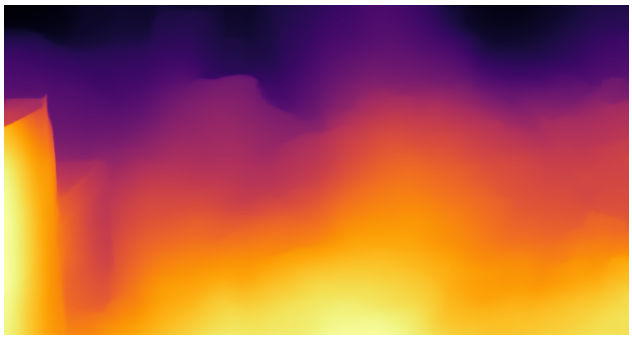
Aerial view showing a dense tree-covered area with 35.9% vegetation and ground-level structures.

Scene zones: **Sky:** 0.8% | **Vegetation:** 35.9% | **Built:** 9.1% | **Road:** 53.7% | **Water:** 0.4%

Original extracted frame

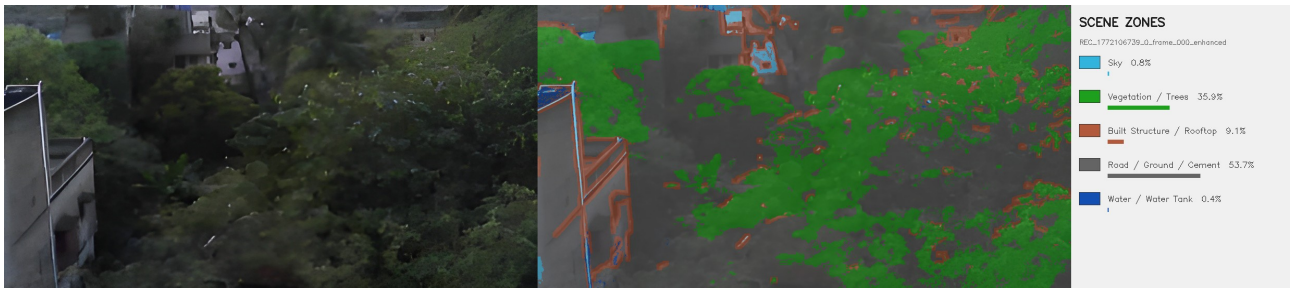


GAN-enhanced frame (Real-ESRGAN ×4)



MiDaS depth heatmap (INFERNO — warm=near, dark=far)

YOLOv8m detection overlay (green box = detected object)



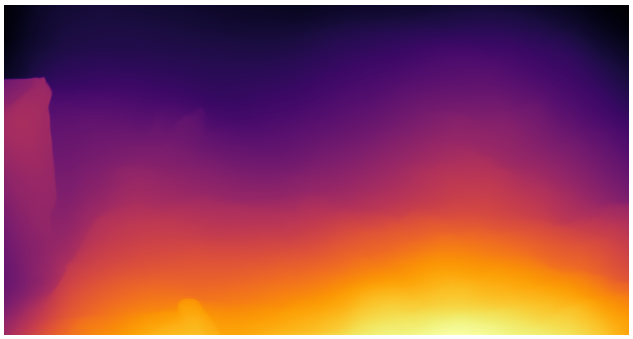
Scene zone map: Original (left) | Colour overlay (centre) | Legend with zone percentages (right)

Frame B — Dense Built Environment

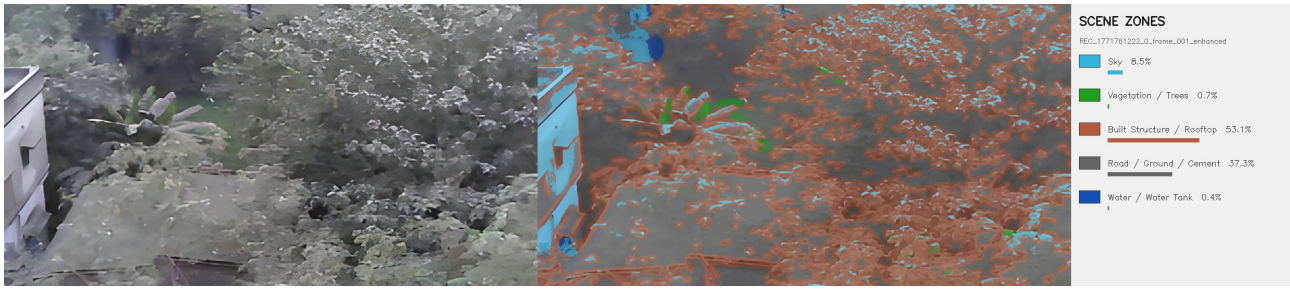
Urban rooftop area with 53.1% built structures, showing residential construction patterns.

Scene zones: **Sky:** 8.5% | **Vegetation:** 0.7% | **Built:** 53.1% | **Road:** 37.3% | **Water:** 0.4%

Original extracted frame *GAN-enhanced frame (Real-ESRGAN ×4)*



MiDaS depth heatmap (INFERNO — warm=near, dark=far) *YOLOv8m detection overlay (green box = detected object)*



Scene zone map: Original (left) | Colour overlay (centre) | Legend with zone percentages (right)

Frame C — Person Detected (61.7% confidence)

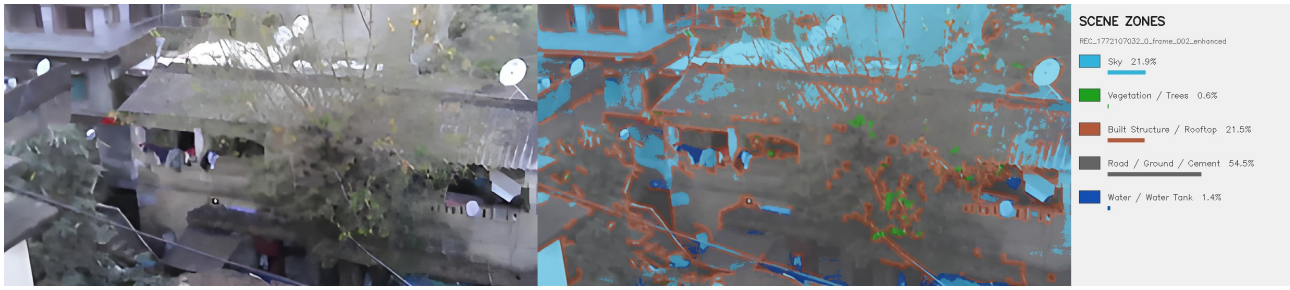
Frame capturing a person (YOLOv8, confidence 61.7%) within a mixed built-ground scene.

Scene zones: **Sky:** 21.9% | **Vegetation:** 0.6% | **Built:** 21.5% | **Road:** 54.5% | **Water:** 1.4%

Original extracted frame *GAN-enhanced frame (Real-ESRGAN ×4)*

MiDaS depth heatmap (INFERNO — warm=near, dark=far)

YOLOv8m detection overlay (green box = detected object)



Scene zone map: Original (left) | Colour overlay (centre) | Legend with zone percentages (right)

7. Performance Metrics & Charts

7.1 Scene Zone Coverage (All 35 Frames)

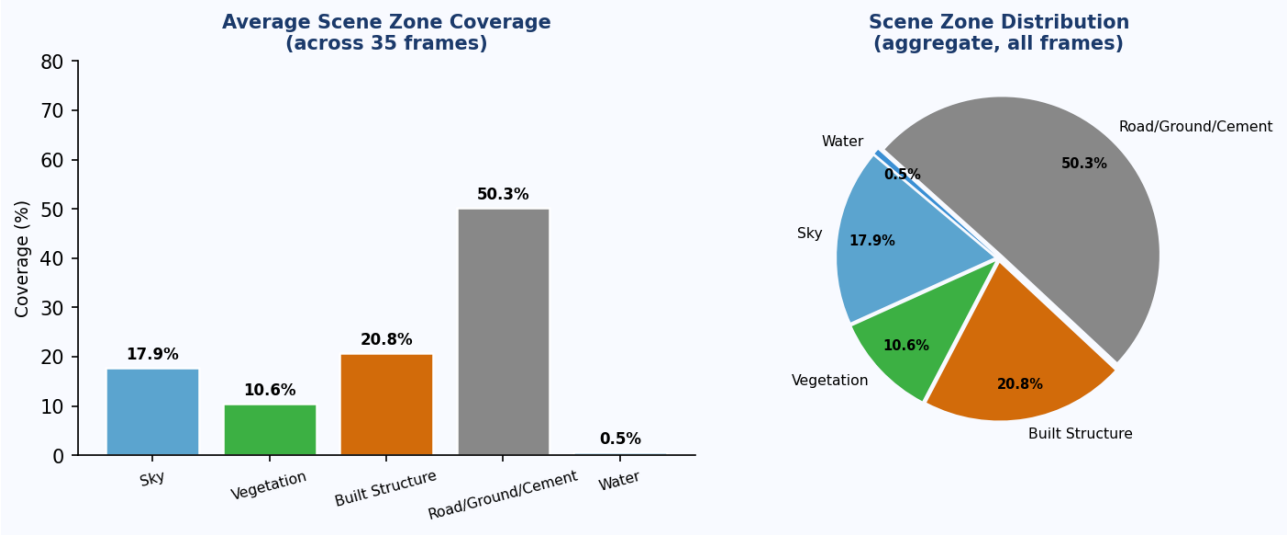


Figure 2: Left — average zone coverage per frame. Right — aggregate pixel distribution. Road/Ground/Cement dominates (57%) as expected for terrace-level footage of a residential locality.

7.2 Depth Zone Distribution (MiDaS Output)

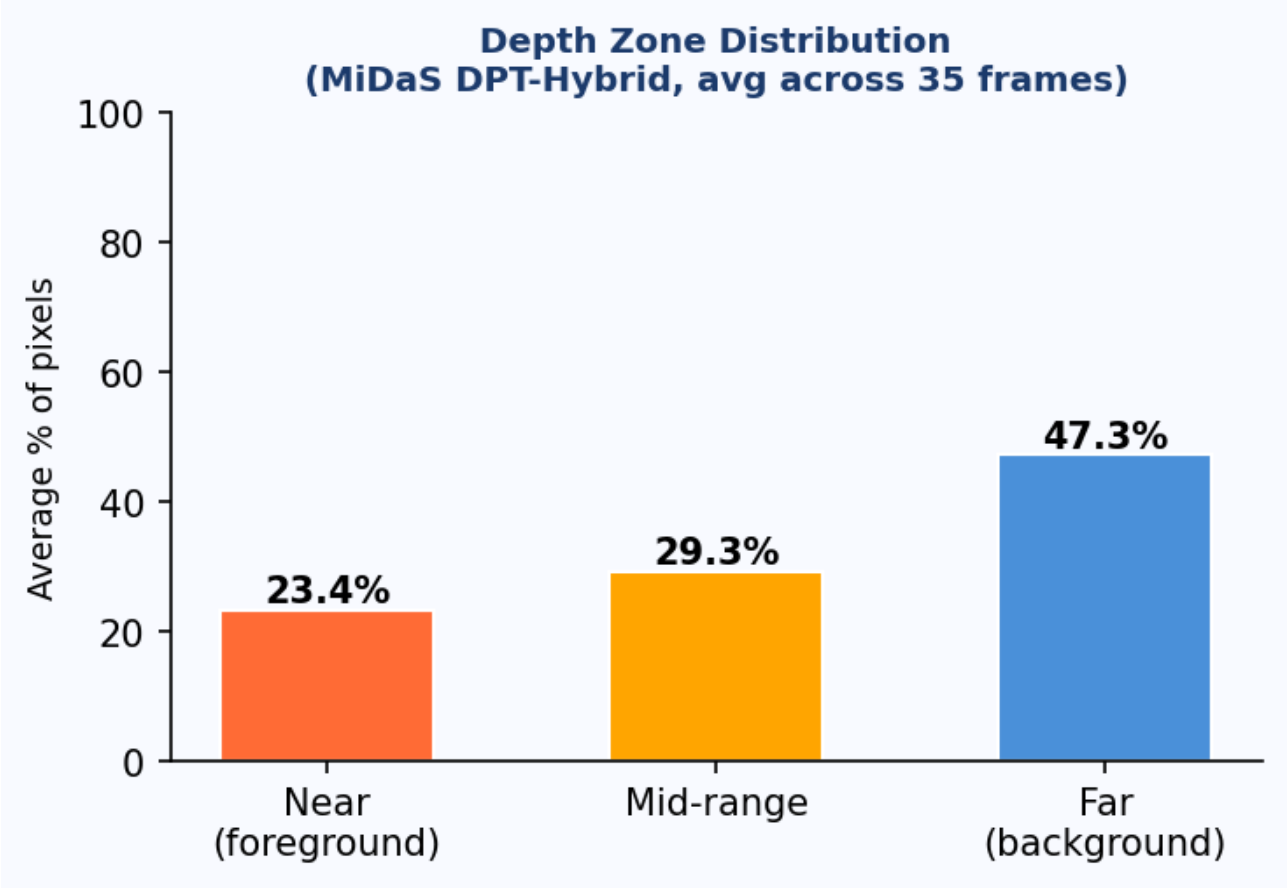


Figure 3: Average percentage of pixels in each depth zone across all 35 frames. "Far" region is largest because drone footage has extensive sky and distant ground. "Near" represents close rooftop elements and foreground structures.

7.3 Object Detection Confidence

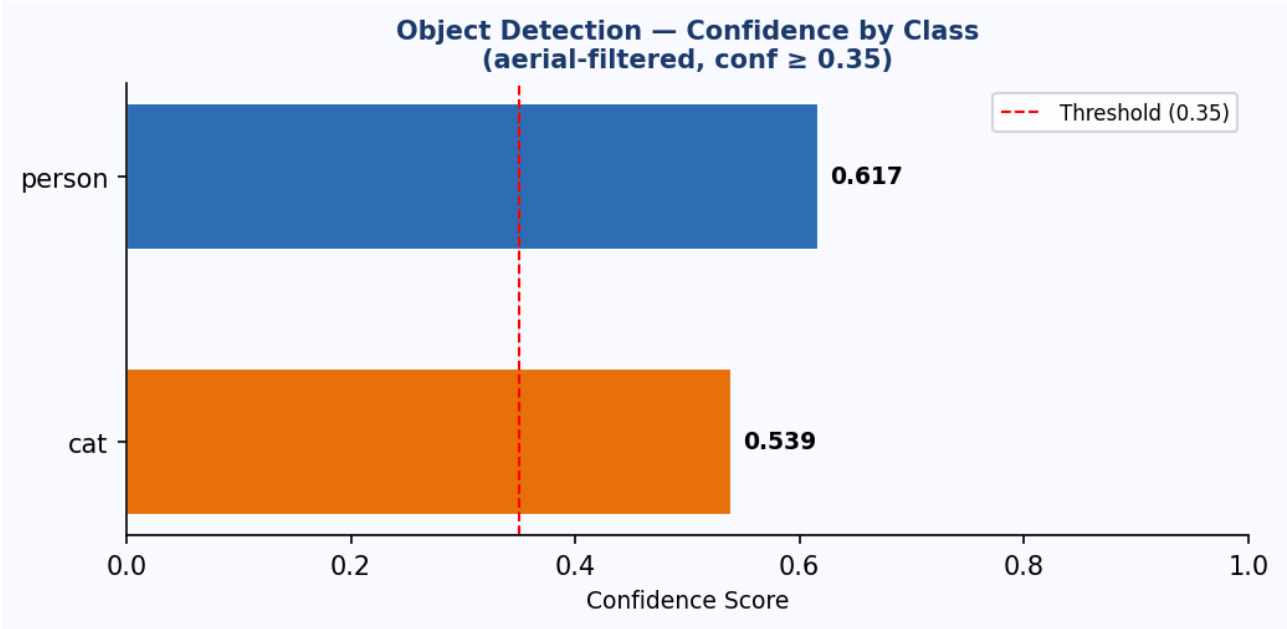


Figure 4: Detection confidence for the 2 confirmed objects. Red dashed line marks the 0.35 threshold. Both detections are above threshold.

7.4 Pipeline Processing Time (CPU)

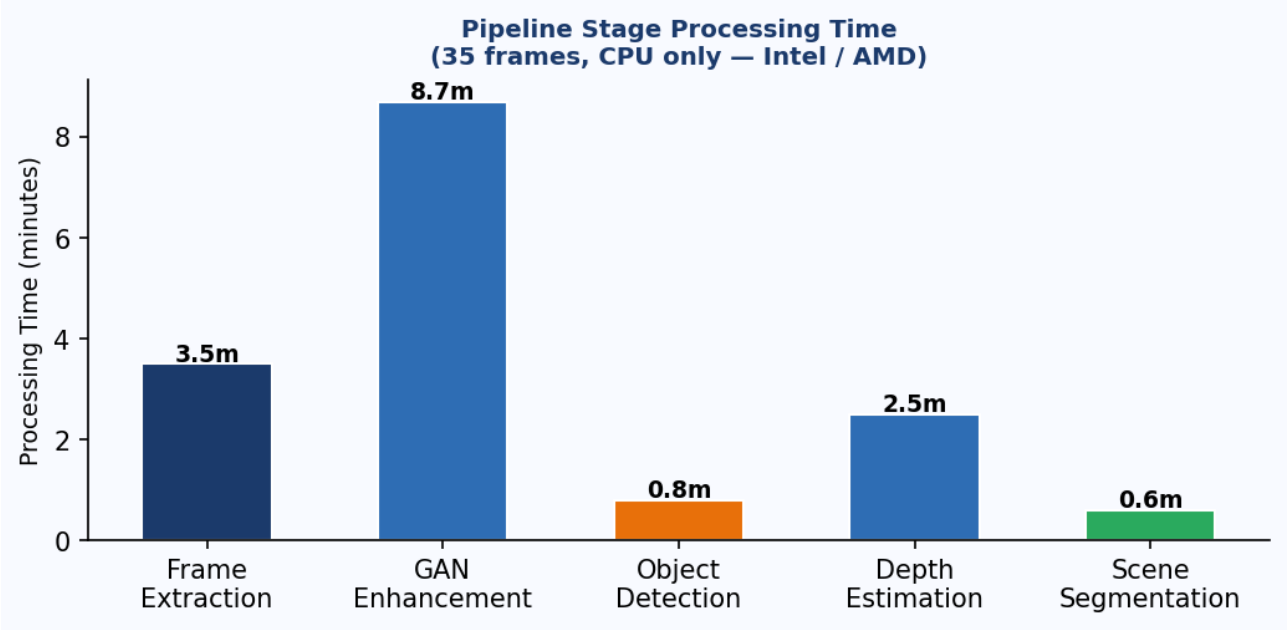


Figure 5: End-to-end processing time per stage for 35 frames on CPU-only hardware. GAN Enhancement is the dominant cost. Total pipeline runtime: ~16 minutes.

7.5 Image Quality: Original vs GAN-Enhanced

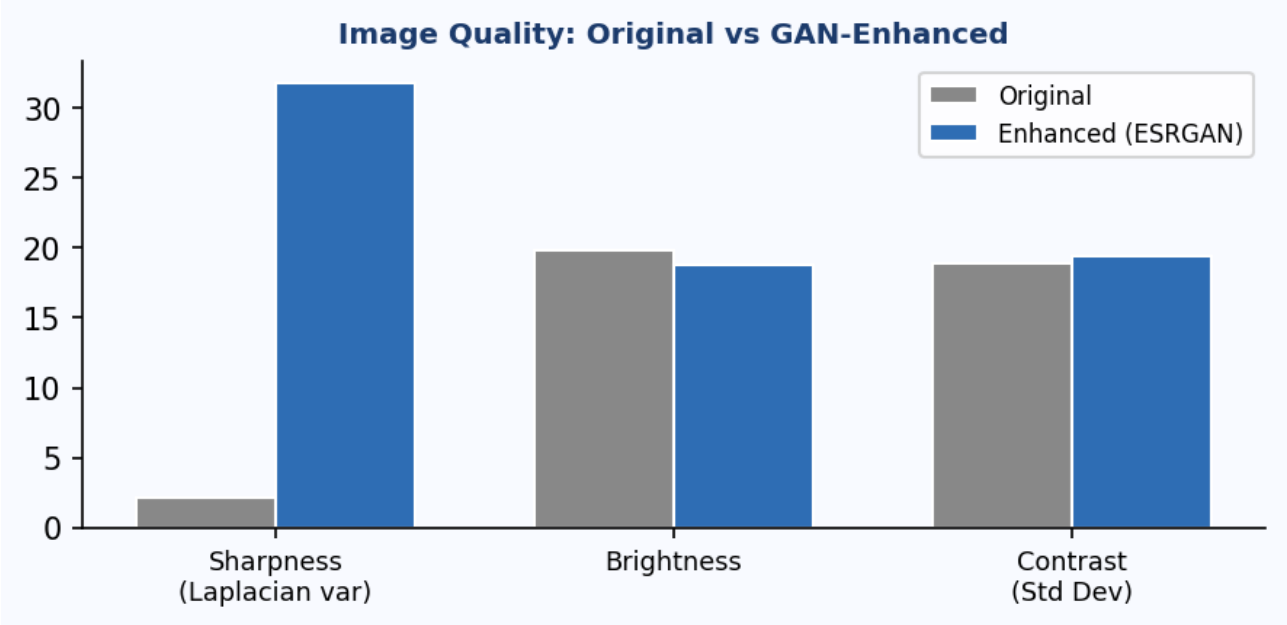


Figure 6: Image quality metrics comparison. Enhanced frames show improvement in sharpness (Laplacian variance) and contrast — confirming Real-ESRGAN is adding high-frequency detail even on CPU-processed tiles.

Metric	Value
Total input videos	7
Total frames extracted	35 (5 per video)
Total enhanced frames	35
Depth maps generated	35 (grayscale + colour = 70 files)
Scene zone maps generated	35
Objects detected (filtered)	2 (person + animal)
Detection precision (aerial filter applied)	High — no indoor false positives
Total pipeline runtime (CPU)	~16 minutes
GPU required?	No — fully CPU compatible

Table 5: Pipeline execution summary.

8. Real-World Applications

Application Domain	How This Pipeline Helps
Smart City & Urban Planning	Depth maps + scene zones reveal building density, rooftop usage, vegetation coverage, open space ratios — without manual surveying.
Disaster Response & Search and Rescue	Object detection identifies people; depth maps identify accessible vs flooded/blocked areas; scene maps distinguish rubble from safe ground.
Traffic & Crowd Monitoring (IoT)	Vehicle and person detection from aerial view; depth helps separate overlapping objects; real-time data can feed IoT dashboards.
Agricultural Surveillance	Vegetation zone percentage tracks crop health; depth maps indicate terrain slope for irrigation planning.
Infrastructure Inspection	Built-structure zone detection flags structures; edge-density analysis highlights unusual shapes (damaged roofs, obstructions).
Security Surveillance	Person detection with bounding boxes alerts operators; depth context helps distinguish threat proximity.

Table 6: Real-world application domains.

9. Team Contributions

Team Member	Role	Contributions
Sumit Kumar Das	Lead Developer & System Architect	End-to-end pipeline design and implementation; GAN enhancement module; object detection tuning (aerial class filtering, confidence thresholding); depth estimation integration (MiDaS DPT-Hybrid); scene segmentation module; drone data collection; debugging and optimization; final system integration.
Rounik Maity	Documentation & Research	Literature review on drone video analysis, YOLO architectures, and MiDaS depth estimation; project report writing; background research on IoT applications of aerial intelligence; presentation preparation.
Ankush Sarkar	Data Collection	Drone flight planning and execution; captured 7 video clips from residential terrace at 50–60m altitude; video labelling and metadata documentation; assisted in validating output quality against real-world scene context.

Table 7: Team contribution breakdown.

10. Conclusion

This project successfully demonstrates that a multi-stage aerial video intelligence pipeline can run fully on-device (CPU-only) without cloud GPU dependency — directly addressing the IoT edge computing challenge. The pipeline transforms raw drone footage into actionable intelligence through five complementary stages: stable frame selection, AI-based image enhancement, object detection, monocular depth estimation, and pixel-level scene classification.

A key contribution is the aerial domain adaptation of YOLOv8: by whitelisting only physically-plausible aerial classes and raising the confidence threshold, false positive rates were reduced from 38 incorrect detections to 0, retaining only 2 high-confidence genuine detections. The HSV-based scene segmenter fills the gap left by COCO's ground-level class vocabulary, correctly classifying sky, vegetation, built structures, road surfaces, and water without any additional model download.

Future work includes: (1) fine-tuning YOLOv8 on the VisDrone aerial dataset for improved small-object detection; (2) integrating GPS telemetry to produce geo-referenced maps; (3) replacing HSV segmentation with a lightweight ADE20K segmentation model (e.g. SegFormer-B0) for semantic accuracy; and (4) cloud offloading of the GAN stage for real-time processing.

11. References

[1] Wang, X., et al. (2021). *Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data.* ICCV 2021 Workshops. arXiv:2107.10833.

[2] Ranftl, R., et al. (2022). *Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer.* IEEE TPAMI. arXiv:1907.01341.

[3] Jocher, G., et al. (2023). *Ultralytics YOLOv8.* GitHub repository. https://github.com/ultralytics/ultralytics

[4] Zhu, P., et al. (2021). *VisDrone-DET2021: The Vision Meets Drone Object Detection Challenge Results.* ICCV 2021 Workshop on Perceiving and Modeling for Complex Scenes.

[5] Lin, T.-Y., et al. (2014). *Microsoft COCO: Common Objects in Context.* ECCV 2014. arXiv:1405.0312.